SOFTWARE REQUIREMENTS SPECIFICATION

ThermaGuard

Project
Urban heat monitoring and intervention planning dashboard

Document status
Project SRS - implementation-aligned draft

Version
1.0 | 11 October 2026

Platform
React/Vite frontend, FastAPI service, SQLite demo persistence, local AWS SAM simulator

Purpose
This document describes the observable scope, requirements, interfaces, constraints, and limitations of the ThermaGuard college/hackathon MVP. Requirements distinguish implemented API capabilities from illustrative dashboard data so the SRS does not imply verified citywide heat measurements.

Prepared from the project source code and configuration. This describes the current implementation, not a claim of production readiness.

Document guide

1. Introduction
2. Overall description
3. System features and functional requirements
4. External interface requirements
5. Non-functional requirements
6. Data, assumptions, and limitations
7. Acceptance criteria

1. Introduction

1.1 Purpose. Define the software requirements for ThermaGuard, a dashboard concept for exploring urban heat information and planning candidate heat mitigation actions. This SRS is intended for students, reviewers, developers, and evaluators.

1.2 Scope. The application provides a multi-page heat dashboard, location/zone exploration, current weather lookup, illustrative alert and analytics views, and an intervention planning workflow. A FastAPI backend provides pilot-zone data, Open-Meteo current weather, a cautious explainable risk response, scenario cost evaluation, and saved action plans.

1.3 Product name. ThermaGuard (backend API title currently CoolBlock API).

1.4 Definitions. Zone: named geographic pilot area. **DEMO data:** illustrative or sample values. **Scenario:** proposed interventions and quantities evaluated against a budget. **SAM local:** local simulation of a Lambda-style API; it does not publish a cloud service.

2. Overall description

2.1 Product perspective. ThermaGuard is a browser-based single-page application paired with a JSON API. The frontend calls relative `/api` paths; Vite proxies them to FastAPI on port 8000 during development. The API may be run directly with Uvicorn or locally in a Lambda simulator using AWS SAM and Docker.

2.2 Intended users. Municipal planners or project reviewers exploring zones and considering budget-feasible interventions; project administrators configuring display preferences; hackathon evaluators reviewing the prototype.

2.3 Operating environment. Modern desktop/mobile browser; frontend Node.js/npm development toolchain; Python 3.13 FastAPI backend; SQLite for local saved plans; optional Docker Desktop and AWS SAM CLI for local Lambda simulation.

2.4 User journey. Start backend, start frontend, open the Vite URL, inspect overview, explore zones and weather, review illustrative alerts/reports, then configure and evaluate an intervention plan.

2.5 Design constraints. The user interface is a dashboard with sidebar navigation. Development API uses HTTP on localhost. The current SAM template uses a SQLite file under the Lambda container temporary directory, so saved plans are demonstration-only and may not survive container replacement.

3. System features and functional requirements

Requirements use “shall” for intended observable behavior. “Implemented” means supported by the inspected application code; illustrative displays are explicitly identified.

ID	Requirement	Status / note
FR-01	The system shall display a dashboard with Overview, Heat explorer, Intervention planner, Heat alerts, Analytics & reports, and Settings views.	Implemented in frontend navigation.
FR-02	On startup, the frontend shall check API health and request pilot zones and supported interventions, then indicate backend connectivity.	Implemented; status shown in top bar.
FR-03	The Overview shall present summary metrics, zone selection, and temperature-profile visualization.	Implemented; summary figures and history are illustrative frontend data.
FR-04	The Heat explorer shall let users search/filter zones, select a zone, inspect a map/panel, and navigate to planning.	Implemented; displayed risk scores/map data are illustrative.
FR-05	For selected coordinates, the system shall request current temperature, apparent temperature, and relative humidity from Open-Meteo through the backend.	Implemented; values can be unavailable; API reports null rather than fabricating.
FR-06	The Alerts view shall show severity-filtered alerts, allow dismissal, and link toward zone exploration/planning.	Implemented as frontend demo content; not a live alert service.
FR-07	The Analytics & reports view shall display zone/forecast summaries and support the available report/CSV/QR actions.	Implemented with illustrative source data; report is generated client-side.
FR-08	Settings shall allow city selection, Celsius/Fahrenheit display preference, and an alert-threshold preference for the current session.	Implemented; settings are not persisted across reloads.
FR-09	The system shall expose a server-owned catalog of supported interventions and illustrative unit costs in INR.	Implemented by GET /api/interventions.
FR-10	Users shall submit a zone, positive budget, and one or more unique intervention IDs with positive quantities for scenario evaluation.	Implemented by POST /api/scenarios; errors identify invalid zone/intervention/input.
FR-11	Scenario evaluation shall calculate estimated cost, remaining budget, feasibility, qualitative benefit, and limitations; it shall not claim a numerical cooling or risk reduction.	Implemented; costs and benefits are planning assumptions.
FR-12	Users shall save a named evaluated action plan; the backend shall recompute cost and feasibility before persistence.	Implemented by POST /api/plans; client-derived figures are not trusted.
FR-13	Users shall retrieve saved action plans.	Implemented by GET /api/plans; persistence is local SQLite in this setup.
FR-14	The API shall report insufficient data rather than calculate a heat-risk score when required validated heat indicators are missing.	Implemented by GET /api/zones/{zone_id}/risk.

3.1 Backend data behavior

Zone records carry coordinates, land-use descriptions, provenance/status, and nullable heat indicators. Current pilot zones are marked DEMO; satellite-derived land-surface temperature, vegetation, shade, and vulnerability indicators are not presently populated. Current weather is retrieved from Open-Meteo. The risk endpoint returns status, methodology and missing-indicator explanations; with current data it can return `insufficient_data` and a null score.

4. External interface requirements

4.1 User interface. The web UI shall support mouse/touch interaction, navigation among the listed pages, readable charts/tables, and responsive layout. It shall communicate backend availability and surface API errors where applicable.

4.2 API interface. The service uses JSON over HTTP. The documented routes are:

Method / path	Purpose
GET /	API welcome response.
GET /api/health	Service health and environment status.
GET /api/zones	Compact pilot-zone list.
GET /api/zones/{zone_id}	Zone detail, provenance, and heat indicators.
GET /api/weather?lat={lat}&lon;={lon}	Current weather for validated coordinates.
GET /api/zones/{zone_id}/risk	Explainable risk result or insufficient-data status.
GET /api/interventions	Supported intervention catalog and illustrative costs.
POST /api/scenarios	Evaluate interventions against a zone budget.
POST /api/plans	Evaluate and save a named action plan.
GET /api/plans	List saved action plans.

4.3 Error behavior. The API returns standard HTTP errors for unknown zones (404), invalid inputs or interventions (422), and upstream weather failures as appropriate. The frontend displays errors for failed requests where handled.

4.4 External services. Current weather depends on Open-Meteo network availability. Local SAM execution depends on Docker and AWS SAM CLI. No AWS account is required for local simulation.

5. Non-functional requirements

ID	Requirement
NFR-01	The app shall run locally for demonstration without provisioning cloud resources.
NFR-02	API responses shall use validated request/response schemas and deterministic cost arithmetic for a given scenario request.
NFR-03	The API shall not fabricate missing environmental measurements or unsupported quantified risk reductions.
NFR-04	The API shall apply configured CORS origins; local defaults include Vite development origins.
NFR-05	The UI shall keep common navigation and page interactions available in a modern browser at desktop and mobile widths.
NFR-06	The backend health endpoint shall return a simple status response for connectivity checks.
NFR-07	Saved-plan persistence shall use SQLite in the current local configuration; SAM demo storage is ephemeral.
NFR-08	The current project is a prototype and shall not be represented as an operational emergency alert or public-safety system.

6. Data, assumptions, and limitations

6.1 Data classification. Backend zone metadata has provenance fields and a DEMO/VERIFIED status. Current bundled pilot data is DEMO. Dashboard metrics, forecast profiles, alerts, and several city/zone visualizations are frontend illustrative content. The app combines those views with live current weather when the weather endpoint is reachable; these sources should not be confused.

6.2 Scenario assumptions. The backend supports native tree planting, public shade structures, and cool roof coating, each with illustrative unit costs in INR. Scenario feasibility means only that the estimated subtotal is within the supplied budget; it is not a procurement quote or measured cooling outcome.

6.3 Risk limitations. A heat-risk score requires validated indicators. Current missing indicators mean the API can decline to provide a score. Frontend sample risk labels are for interface demonstration, not authoritative risk assessment.

6.4 Persistence and deployment. Local SQLite action plans are demonstration data. In the SAM template, the database path is under `/tmp` inside the local Lambda container. The current no-cost setup is local-only and is not a publicly reachable deployment. Running `sam deploy` is outside this SRS scope and provisions AWS resources.

6.5 Assumptions. The operator has Node.js/npm for the frontend and Python dependencies for direct backend execution, or Docker Desktop and SAM CLI for the local Lambda simulation. Internet connectivity is available for Open-Meteo current-weather requests.

7. Acceptance criteria

ID	Acceptance check
AC-01	Frontend starts and the Overview page loads at the local Vite URL.
AC-02	With backend running, topbar reports online and API health returns status ok.
AC-03	Zones and intervention catalog can be retrieved from the API.
AC-04	Selecting a supported zone can request current weather; missing/upstream data is surfaced without inventing readings.
AC-05	A valid scenario returns itemized estimated cost, budget remainder, feasibility, qualitative benefits, and limitations.
AC-06	A named valid plan can be saved and listed; server-derived totals match scenario evaluation.
AC-07	A risk request with missing verified indicators returns insufficient-data status rather than a fabricated score.
AC-08	The UI clearly treats alerts, forecast, and summary indicators as illustrative prototype content.

End of document